

Scaling the Patient Data Refresh Orchestrator from 5K to 10M patients

We run out of endpoint capacity long before we run out of database.

Core conclusion. At 10M patients refreshing daily, the system needs to sustain about 116 refreshes per second. Postgres handles that. The problem is upstream: an endpoint rated at 100 requests per minute gives us 144,000 refreshes a day, so covering 10M patients daily takes roughly 70 of those endpoints running flat out, and that is before status polling eats into the same allowance. Our throughput is whatever the connected endpoints will let us have, and adding servers on our side does not change it.

WHAT IS BEING SCALED

This is the orchestrator from Part 1: TypeScript on Node 22 with Fastify, backed by one PostgreSQL 16 database that acts as the job queue, the scheduler's source of truth and the rate limiter all at once. A scheduler tick inserts one job per due patient. Workers claim jobs atomically with `SELECT ... FOR UPDATE SKIP LOCKED` in priority order, debiting the endpoint token buckets in the same transaction. The worker fires `POST /patients/{id}/$updateData`, hands the job back to the database instead of holding a thread open while the EHR works, and picks up polling on a later tick. On completion it writes `last_refresh_at` for every study obligation that one call covered.

Two things about this design shape everything below.

- **The unit of work is the patient, not the patient-endpoint pair.** The upstream API is patient-level, so a single call refreshes everything we hold for that patient. This is what makes coalescing across studies possible.
- **Admission is all-or-nothing across endpoints.** A claim debits a token from every endpoint holding any of the patient's data, taking the bucket rows in ascending `endpoint_id` order so claims cannot deadlock. A patient can therefore only refresh as fast as their slowest source. That is the right behavior for a patient-level API, and it is also what makes sharding awkward later.

ASSUMPTIONS

I would want to replace each of these with a real measurement before spending engineering time.

1. Everyone refreshes daily. If the real mix is mostly weekly, divide the throughput numbers accordingly; nothing else changes.
2. Endpoint limits of roughly 100 req/min for Epic-class systems and 30 req/min for the rest, per the brief.
3. Retrieval takes about 30 seconds and we poll it about once a second, which is how the Part 1 mock behaves.
4. **Status polls do not count against vendor quota.** Our implementation only debits tokens on the trigger call, and I do not know whether real vendors charge for status reads. If they do, real capacity drops by roughly the poll multiple, so 20–30x. Of everything here, this assumption moves the numbers most.
5. The patient count grows but studies and endpoints per patient stay about the same. More enrollments per patient would make coalescing worth more and the scheduler sweep worse.

WHAT THE NUMBERS SAY

Scale / parameter	Demand	Implication
5K patients, daily	0.06 refreshes/sec	Roughly three orders of magnitude of headroom in the current design
10M patients, daily	116 refreshes/sec	About 70 endpoints at 100 req/min running continuously, or about 230 at the 30 req/min tier
Job-state writes	~500 writes/sec	Three row writes per job plus about 1.5 patient_studies updates per call; one tuned primary carries this
30s retrieval, 1s poll	~30 polls per refresh	Another ~3,500 writes/sec if persisted, and if polls are charged against quota, about 97% of API spend is polling

The database numbers are big but unremarkable. The endpoint number is the one we cannot engineer our way out of, and it sets the order of work: stop making calls we do not need, clear the internal hot spots that keep us from using the capacity we

have, make eligibility cost scale with what is due, and shard only once a measurement shows a single primary is saturated. That last point is the one I would defend hardest. Do not start with sharding. Everything above it is cheaper, reversible, and may well make it unnecessary.

BOTTLENECKS, FIXES, AND THE ORDER TO DO THEM IN

#	Bottleneck	Change	Trade-off and trigger
1	Full enrollment sweep each tick	Persist and index <code>next_due_at</code> with per-patient jitter; shard scheduler by patient hash	A denormalized column to maintain and reconcile; act when tick duration approaches the tick interval
2	Multi-bucket claim contention	Lease token blocks to workers; stripe the counter if it stays hot	Leased-but-unspent quota is lost throughput; act when claim p99 rises with worker count
3	Poll amplification	Webhooks first, then adaptive intervals and batch status endpoints	Mostly a vendor contract question; act when polls dominate API spend
4	Terminal job history	Partition <code>refresh_jobs</code> by time, archive cold partitions	Ongoing partition maintenance; at 10M daily this is day one, not a later trigger
5	Permanent failures retried indefinitely	Terminal dead-letter state after N permanent failures, with a named owner	Needs a downstream owner for stale patients; act once volume exceeds human review
6	Connection pressure	PgBouncer transaction pooling	Session-local state has to go; act as connections approach <code>max_connections</code>
7	Single-write-primary ceiling	Shard by endpoint / failure domain	Cross-endpoint patients get hard; do this last, and only on evidence

Each of these ships on its own, and each waits on a measurement rather than a forecast, cheapest and most reversible first. In practice steps 1 through 4 should buy enough headroom that step 7 never comes up.

THE TWO CHANGES THAT MATTER MOST

Make eligibility cost what is due, not what exists

`findDuePatients` checks `last_refresh_at + refresh_interval <= now()` against every active enrollment. At 10M patients that is a full scan of roughly 15M rows on every tick, and since the predicate is an expression spanning two tables, no index helps. The fix is to store the answer rather than recompute it: an indexed `next_due_at` on `patient_studies`, written when a job completes. The sweep becomes a range scan sized by what is actually due.

Two things have to come with it. The column has to be updated on enrollment creation, cadence changes and consent changes as well as on completion, with a reconciliation job to catch drift, since a stale `next_due_at` is a patient who quietly stops refreshing and nobody notices. And it needs deterministic per-patient jitter, since an unjittered daily sweep at 10M patients is a ten-million-request stampede at midnight.

Bucket contention is worse than an ordinary hot row

Every claim takes a lock on each of the patient's endpoint buckets. That is deadlock-free, but it will not survive adding workers, and for a worse reason than normal row contention. Because a claim locks every endpoint the patient touches, two claims for completely unrelated patients serialize as long as those patients share a single endpoint. If our endpoints are a handful of big national systems, almost every claim in the system fights over the same few rows, and throughput falls toward what one row can do.

The fix is to get the bucket off the claim path. A worker leases `k` tokens for an endpoint, holds them for a few seconds and spends them locally, so the all-or-nothing check becomes a local question: do I hold a token for every endpoint this patient touches right now? That is one row touch per lease instead of one per job, roughly 50–100× less contention at `k=50`. The trade is real: leased tokens that expire unspent are capacity we were allowed to use and did not, and tokens held by a crashed worker sit idle until the lease runs out. Leases of one to five seconds cap both losses.

SHARDING, AND WHY IT FIGHTS THE ADMISSION RULE

Endpoint is the obvious axis to shard on. It is already the quota domain, the failure domain and the outage domain, so a shard would own a self-contained problem: its own queue, capacity, circuit breaker and blast radius.

The catch is worth stating plainly. All-or-nothing admission needs an atomic debit across every endpoint holding a patient's data. Shard by endpoint and that single-database transaction becomes a distributed one, for exactly the multi-endpoint patients that make patient-level coalescing worth having in the first place. There are three ways out. Route by primary endpoint and give up all-or-nothing admission, which is cheap but weakens the "one call refreshes everything" guarantee and lets one shard admit a call another has no quota for. Keep a small global admission service for multi-endpoint patients only, which preserves the semantics but puts a coordination point back on the hot path. Or shard by patient and replicate quota state, which just moves the hot counter somewhere else.

Which one is right depends on a number I do not have: what share of patients actually span more than one endpoint. If it is small, route by primary endpoint. If it is large, the thing to reconsider is the admission rule itself rather than the sharding topology, because tying every patient to their slowest source gets expensive at 10M patients however the data is split.

AVOIDED CALLS ARE RECOVERED CAPACITY

Throughput is capped by the endpoints, so a call we avoid is not just money saved. It is capacity back.

- **Coalescing across studies.** Because the unit of work is the patient rather than the patient-study pair, a patient with three studies falling due at once costs one API call instead of three. The part that matters at 10M is the consequence: API spend tracks how many patients we refresh, not how many enrollments exist. Studies per patient can grow, and new partners can enroll into the same population, without costing any endpoint capacity. Capacity is the one thing we cannot buy more of, so this property is worth protecting when we make later decisions, particularly sharding.
- **Adaptive frequency.** If a patient's data has not changed in ten consecutive refreshes, refresh them less often. Needs reliable change detection and clinical sign-off on a longer staleness window, so it is a product decision rather than one engineering makes alone.
- **Demand-driven refresh.** Do not refresh data nobody reads. Refresh on first access within a staleness bound and accept a slow first read. Probably the biggest saving available and the biggest change in behavior.
- **Backpressure.** Cap live jobs per endpoint at what its quota can drain in N minutes. An unbounded queue in front of a fixed-rate resource does not add throughput, just latency.

WHAT DOES NOT CHANGE

The four core operations hold their shape at 10M, and so do most of the decisions underneath them. Postgres now() stays the only clock, because with two clocks you get failure modes nobody can reproduce. The idempotency key stays (job id, attempt): stable across a crash, new on a real retry, which is what makes at-least-once delivery safe when every call costs money. The transient/permanent/throttled split is what gives retry policy anything to work with. Skipping heartbeats still makes sense, since one unit of work is one bounded HTTP call and a fixed lease above the HTTP timeout covers it; the only thing scale adds is an index for the lease-expiry sweep.

Two reliability pieces do need adding. A per-endpoint circuit breaker, so that millions of jobs do not each independently discover that Epic is down. And priority aging, since starving low-priority refreshes is theoretical at 5K and certain at 10M. I would not move the queue to Kafka: it gives ordered partitioned streaming, and this system needs per-item mutable state, retry budgets, priority reordering and quarantine, all of which we would rebuild in a database sitting next to it.

WHAT I WOULD WANT TO KNOW FIRST

1. **How many endpoints are we talking to, and how big are they?** Ten large systems and ten thousand small ones are very different problems, and it decides whether we are rate-limit-bound at all.
2. **Will any vendor push a completion webhook?** Changes the architecture more than anything we can do on our own side.
3. **What share of patients span more than one endpoint?** Decides the sharding design, and whether all-or-nothing admission survives.
4. **What is the actual freshness requirement?** Protocol "daily" and SLA "daily" are often not the same thing, and the difference is capacity.
5. **What happens downstream when a refresh fails for good?** At 10M, a 2% failure rate is 200,000 patients with silently stale records. A clinical problem before an engineering one.